

Lecture 19: P, NP, and NP-Completeness

*Instructor: Sourav Chakraborty**Scribe: Sagnik Bhattacharyya, Bimit Mandal*

1 Decision Problems and Languages

Throughout this course we have studied problems such as shortest paths, minimum spanning trees, and network flow, all of which admit efficient algorithms. We now ask: are there natural, well-stated problems for which no efficient algorithm can exist? To answer this rigorously we first fix a common format for problems.

Definition 1 (Language / Decision Problem). *Let Σ be a finite alphabet. A language is any set $L \subseteq \Sigma^*$. The associated decision problem is: given $x \in \Sigma^*$, determine whether $x \in L$.*

Encoding a combinatorial problem as a language is standard. Two representative examples are:

- $L_{\text{VERTEXCOVER}} = \{(G, k) \mid G \text{ has a vertex cover of size } \leq k\}$.
- $L_{\text{SP}} = \{(G, s, t, k) \mid \text{the shortest } s\text{-}t \text{ path in } G \text{ has length } \leq k\}$.

For L_{SP} we know a polynomial-time algorithm (Dijkstra / BFS); for $L_{\text{VERTEXCOVER}}$ no such algorithm is known.

2 The Complexity Class P

Definition 2 (The class **P**). ***P** is the set of all languages $L \subseteq \Sigma^*$ for which there exists a "deterministic" Turing Machine A and a constant $c \geq 1$ such that*

- $L(A) = L$, where $L(A)$ is the language accepted by the Turing Machine A and A halts for any input $x \in \Sigma^*$.*
- Given an initial input $x \in \Sigma^*$, A runs for at most $O(|x|^c)$ steps before halting*

Informally, **P** is the class of problems that are efficiently solvable. The polynomial bound is robust: it is closed under composition, and polynomials grow far more slowly than the exponential search spaces that brute-force would require.

3 The Complexity Class NP

For many problems, finding a solution seems hard, but checking a proposed solution is easy. This asymmetry motivates the class **NP**.

Definition 3 (The class **NP** — verifier formulation). **NP** is the set of all languages L for which there exists a polynomial-time "non-deterministic" Turing Machine T (the verifier) which accepts L .

The definition can also be written in terms of deterministic Turing Machines:

A language $L \subset \Sigma^$ is in **NP** iff there exists a deterministic turing machine T , which satisfies the following constraint:*

- (i) For any input (x, y) T runs for at most $p(|x|)$ many steps before halting where p is a polynomial*
- (ii) $\forall x \in L$, there exists a string $y \in \Sigma^*$ of length $q(|x|)$, where q is a polynomial, such that T accepts (x, y)*
- (iii) For any $x \notin L$, given any $y \in \Sigma^*$, T doesn't accept (x, y) .*

The string y for which T accepts (x, y) for some $x \in L$ is oftentimes called the witness for $x \in L$.

Remark: To give an example of an **NP** language, consider the language,

$$\mathbf{HAMPATH} = \{G \mid G \text{ is a finite simple graph that has a hamiltonian path}\}$$

Note that we can encode the graph G by either enumerating all possible finite simple graphs G (since there are countably many of them) and assign a Turing Machine M which will convert any integer to the corresponding adjacency list, or we can just use the adjacency list encoded in binary as input. To see why **HAMPATH** is in **NP**, notice that a possible witness for can be a hamiltonian path in G . Given a graph G with m edges and n vertices, an adjacency list encoding of G will require $O(nm \log n)$ many bits as input. The witness will need $O(n \log n)$ many bits. A verifier Turing machine can just traverse through the input witness as a path and verify the truthfulness of the statement in $O(n \log n)$ time.

Fact 1. $\mathbf{P} \subseteq \mathbf{NP}$.

Proof. If $L \in \mathbf{P}$ then A decides L in polynomial time without any witness; set $\omega = \varepsilon$ and use A as the verifier. $\square$

Conjecture 3.1 ($\mathbf{P}$ vs. $\mathbf{NP}$). $\mathbf{P} \neq \mathbf{NP}$.

This is the central open problem of theoretical computer science. While it is unproven, virtually all evidence and expert opinion points toward $\mathbf{P} \neq \mathbf{NP}$.

4 Boolean Satisfiability (SAT)

A central language in **NP** is the satisfiability language SAT.

Definition 4 (Satisfying Assignment). Let $\varphi(x_1, x_2, \dots, x_n)$ be a Boolean formula over variables $x_1, x_2, \dots, x_n \in \{0, 1\}$. A **satisfying assignment** is a truth assignment $\sigma : \{x_1, \dots, x_n\} \rightarrow \{0, 1\}$ such that:

$$\varphi(\sigma(x_1), \sigma(x_2), \dots, \sigma(x_n)) = 1$$

Definition 5 (CNF). A **CNF** is a boolean formula which is of the form,

$$C_1 \wedge C_2 \wedge \cdots \wedge C_m$$

where each C_i of the form,

$$C_i = \bigvee l_{i,j}$$

and each **literal** $\ell_{i,j}$ is either a propositional variable x_k or its negation $\neg x_k$, for some variable $x_k \in \{x_1, x_2, \dots, x_n\}$.

Example. The following formula is in CNF:

$$(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2) \wedge (x_2 \vee \neg x_3)$$

Every propositional formula can be converted into an equivalent CNF formula via the application of De Morgan's laws and the distributive law.

Definition 6 (SAT). The **Boolean Satisfiability Language (SAT)** is defined as follows:

$$\mathbf{SAT} = \{\varphi \mid \varphi \text{ is a } \mathbf{CNF} \text{ which has a satisfying assignment}\}$$

Observation 4.1. $\mathbf{SAT} \in \mathbf{NP}$.

Proof. Given a formula φ and an assignment $\omega = (x_1, \dots, x_n) \in \{T, F\}^n$, we can evaluate $\varphi(\omega)$ in polynomial time. Hence ω serves as the witness. $\square$

Definition 7 (k-SAT). For a fixed integer $k \geq 1$, **k-SAT** is the restriction of SAT in which every clause C_i contains exactly k literals:

$$\varphi = \bigwedge_{i=1}^m (\ell_{i,1} \vee \ell_{i,2} \vee \cdots \vee \ell_{i,k})$$

Remark: 2 – SAT is in **P** while k – SAT for $K \geq 3$ is in a class called **NP-complete** languages. We will discuss this in the following sections.

5 Polynomial-Time Reductions

Definition 8 ($L_1 \leq_p L_2$). L_1 is polynomial-time many-one reducible to L_2 , written $L_1 \leq_p L_2$, if there exists a polynomial-time computable function $\rho : \Sigma^* \rightarrow \Sigma^*$ such that

$$\forall x : x \in L_1 \iff \rho(x) \in L_2.$$

The function ρ is computed in polynomial time (many-one reduction). Reductions transfer difficulty: if $L_1 \leq_p L_2$ and $L_2 \in \mathbf{P}$, then $L_1 \in \mathbf{P}$. Contrapositively, if L_1 is hard and $L_1 \leq_p L_2$, then L_2 is at least as hard as L_1 .

Remark: In lieu of the definition of polynomial time reductions, $\exists c \in \mathbb{N}$ such that $\forall x \in L_1$, $\rho(x)$ must be of length $O(|x|^c)$.

Example: $\text{VertexCover} \leq_p \text{SAT}$. Given (G, k) with $G = (V, E)$, introduce a Boolean variable x_u for each vertex $u \in V$ ($x_u = T$ means u is in the cover). The CNF formula $\varphi_{G,k}$ contains two "gadgets":

$$\mathbf{VertexCover}_G = \bigwedge_{(u,v) \in E} x_u \vee x_v$$

which accounts for the fact that for the set of variables $S \subset \{x_1, \dots, x_n\}$ which are set to 1, the corresponding vertex set is a vertex cover and **k-Vertex** $_G$ which accounts for the fact that at most k variables x_i are set to 1.

Hence, we can define.

$$\varphi_{G,k} = \mathbf{VertexCover} \wedge \mathbf{k-Vertex}$$

We don't give a construction for **k-Vertex** here but the idea is as follows: We design a polynomial size adder circuit to add the binary variables $x_1, \dots, x_n$ (using half-adder and full-adder circuits this will take $O(n)$ many $\vee$ and $\wedge$). We assign separate variables $x_{i,\text{sum}}$ which will store the i th bit of this sum (to write these as clauses we set up the circuit for the i th bit of the sum and use a xor clause to enforce equality with $x_{i,\text{sum}}$).

Next we add the range check condition on the sum by emulating another adder circuit and storing the difference in variables $x_{i,\text{diff}}$. We check the highest set bit $x_{\lfloor \log n \rfloor + 2, \text{diff}}$. If it is 1, then the sum is indeed less than k , else it is greater than k .

Then $(G, k) \in L_{\text{VERTEXCOVER}}$ if and only if $\varphi_{G,k} \in \text{SAT}$. The reduction $\rho(G, k) = \varphi_{G,k}$ is computable in polynomial time, establishing $\text{VERTEXCOVER} \leq_p \text{SAT}$.

We can also reduce $2 - \text{SAT}$ to the problem of strongly connected components in a directed graph G which is solvable in polynomial time.

6 NP-Completeness and the Cook–Levin Theorem

Definition 9 (NP-hard and NP-complete). *A language L is **NP-hard** if $L' \leq_p L$ for every $L' \in \mathbf{NP}$. It is **NP-complete** if additionally $L \in \mathbf{NP}$.*

NP-complete problems are the hardest in **NP**: an efficient algorithm for any one of them would collapse **NP** into **P**.

Theorem 6.1 (Cook–Levin, 1971–1973). *SAT is NP-complete.*

Proof.[Proof sketch] We already observed $\text{SAT} \in \mathbf{NP}$. It remains to show that every $L \in \mathbf{NP}$ polynomial-time many-one reduces to SAT.

Let $L \in \mathbf{NP}$. By definition, there exists a polynomial-time nondeterministic Turing machine (NTM) M with a single accept state q_{acc} , a single reject state q_{rej} , and transition function

$$\delta : Q \times \Gamma \longrightarrow \mathcal{P}(Q \times \Gamma \times \{L, R\})$$

such that for every input $x \in \{0, 1\}^n$:

$$x \in L \iff M \text{ accepts } x \text{ along some computation path within } p(n) \text{ steps,}$$

for some polynomial p .

The Tableau. Fix an input $x \in \{0, 1\}^n$ with $|x| = n$. A tableau for M on x is a $(p(n) + 1) \times p(n)$ table T , where:

- Rows $0, 1, \dots, p(n)$ represent time steps.
- Columns $1, 2, \dots, p(n)$ represent tape cell positions.
- Each entry $T[i][j]$ belongs to $\Gamma \cup (Q \times \Gamma)$, encoding either a tape symbol or a (state, symbol) pair at the position of the read/write head.

Row $T[0]$ encodes the initial configuration: the tape holds x followed by blanks, the head is at position 1, and M is in start state q_0 . Row $T[i+1]$ is obtained from row $T[i]$ by applying one step of δ . A tableau is accepting if the state in some entry of row $T[p(n)]$ is q_{acc} .

Boolean Variables. For each time step $i \in \{0, \dots, p(n)\}$, cell position $j \in \{1, \dots, p(n)\}$, and symbol $s \in \Gamma \cup (Q \times \Gamma)$, introduce a Boolean variable:

$$z_{i,j,s} = 1 \iff T[i][j] = s.$$

The total number of variables is $\mathcal{O}(p(n)^2 \cdot |\Gamma \cup (Q \times \Gamma)|)$, which is polynomial in n .

The Formula φ_x . We construct a CNF formula φ_x as the conjunction of four groups of clauses:

- (i) **Cell legality.** Each cell contains exactly one symbol at each time step:

$$\forall i, j : \left(\bigvee_s z_{i,j,s} \right) \wedge \bigwedge_{s \neq s'} (\neg z_{i,j,s} \vee \neg z_{i,j,s'}).$$

- (ii) **Initial configuration.** Row $T[0]$ encodes the correct start configuration of M on x : the tape reads $x_1 x_2 \dots x_n \sqcup \dots \sqcup$, the head is at position 1, and the state is q_0 :

$$z_{0,1,(q_0,x_1)} \wedge \bigwedge_{j=2}^n z_{0,j,x_j} \wedge \bigwedge_{j=n+1}^{p(n)} z_{0,j,\sqcup}.$$

- (iii) **Valid transitions.** Each pair of consecutive rows $T[i]$ and $T[i+1]$ constitutes a locally valid step of δ . For each window of three consecutive cells $(j-1, j, j+1)$ at step i , the

triple $(T[i][j-1], T[i][j], T[i][j+1])$ must be consistent with some application of δ :

$$\forall i, j : \bigvee_{\text{valid windows}} (z_{i,j-1,a} \wedge z_{i,j,b} \wedge z_{i,j+1,c} \wedge z_{i+1,j-1,a'} \wedge z_{i+1,j,b'} \wedge z_{i+1,j+1,c'}).$$

This is convertible to CNF in polynomial time. Cells not adjacent to the head are copied unchanged.

(iv) **Accepting configuration.** The final row $T[p(n)]$ contains q_{acc} at some position:

$$\bigvee_{j=1}^{p(n)} z_{p(n),j,(q_{\text{acc}}, \cdot)}.$$

Correctness. The formula φ_x is satisfiable if and only if there exists a valid accepting tableau for M on x , i.e., there exists a nondeterministic choice sequence (witness) ω such that M accepts x in at most $p(n)$ steps. Hence:

$$x \in L \iff \varphi_x \in \text{SAT}.$$

Efficiency. The formula φ_x has $\mathcal{O}(p(n)^2)$ variables and $\mathcal{O}(p(n)^2)$ clauses, each of constant width (determined by $|Q|$ and $|\Gamma|$, which are fixed constants of M). The reduction $x \mapsto \varphi_x$ is therefore computable in polynomial time, and constitutes a valid polynomial-time many-one reduction $L \leq_p \text{SAT}$. Since $L \in \mathbf{NP}$ was arbitrary:

$$\forall L \in \mathbf{NP}, \quad L \leq_p \text{SAT},$$

which together with $\text{SAT} \in \mathbf{NP}$ establishes that SAT is NP-complete. $\square$

Corollary 6.2. $\text{SAT} \in \mathbf{P} \iff \mathbf{P} = \mathbf{NP}$.

More generally, since every NP language reduces to SAT, any single NP-complete problem admitting a polynomial-time algorithm would imply $\mathbf{P} = \mathbf{NP}$.

7 Summary

Class / Concept	Informal meaning	Key example
P	Decidable in poly time	Shortest path
NP	Verifiable in poly time	SAT, VERTEXCOVER
NP-hard	Every NP problem reduces to it	SAT
NP-complete	NP-hard $\cap$ NP	SAT, VERTEXCOVER, 3-Coloring
$L_1 \leq_p L_2$	Poly-time many-one reduction	VERTEXCOVER $\leq_p$ SAT

The chain of reductions for establishing further NP-completeness results proceeds by showing $\text{SAT} \leq_p (\text{new problem})$, thereby inheriting hardness from SAT. This is the standard technique introduced by Karp (1972) for showing 21 problems NP-complete.